

Algorithmen und Datenstrukturen

Übung 5: Suchen II

Robert

2026-06-02

Wiederholung QuadTrees

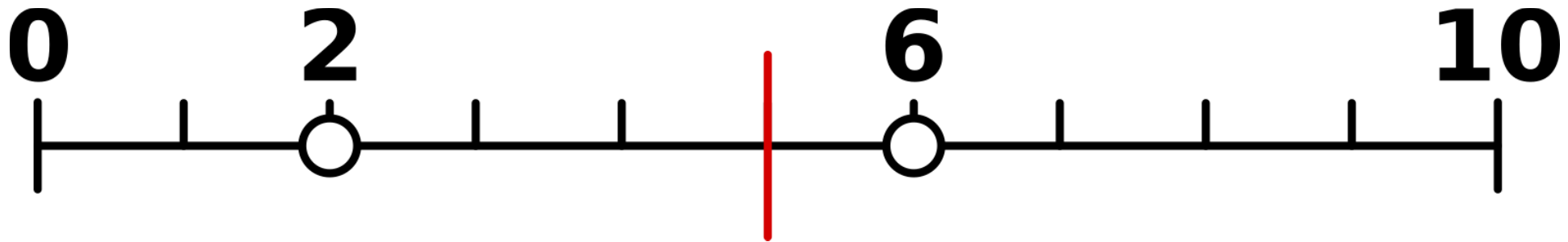
QuadTree

- ein QuadTree ist eine Speicherstruktur für räumliche Daten
- der Datenraum wird in Blöcke aufgeteilt
- bei einem Überlauf in einem Block wird dieser in weitere Blöcke aufgeteilt
 - das wird rekursiv weitergeführt, bis es keinen Überlauf mehr gibt

Es gibt zwei Arten der Aufteilung:

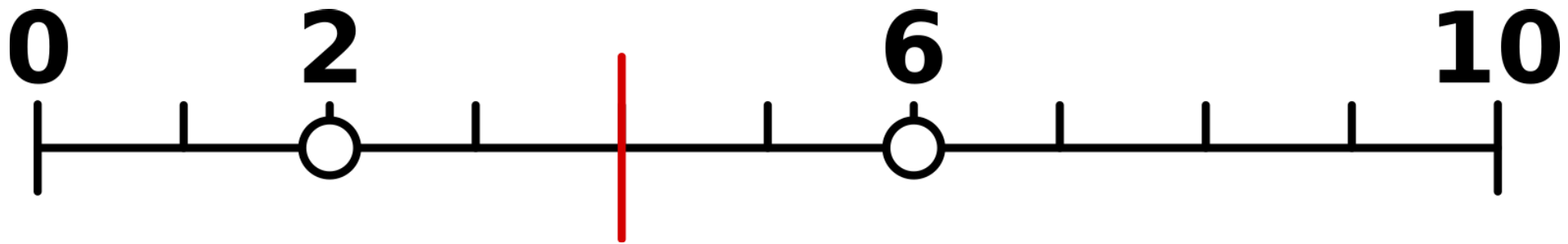
1. Aufteilung mittig im Datenraum

- die Mitte zwischen dem größten und kleinsten Wert im Datenraum wird verwendet
- Beispiel: Der Datenraum reicht von 0 bis 10, daher wird die 5 zur Aufteilung verwendet



2. Aufteilung mittig bezüglich der Datenmenge

- der Durchschnitt aller Daten, die in der Datenmenge innerhalb des Blocks vorkommen, wird verwendet
- Beispiel: Der Durchschnitt der beiden Datenpunkte liegt bei $\frac{2+6}{2} = 4$, daher wird die 4 zur Aufteilung verwendet



Aufgabe 1

1 a)

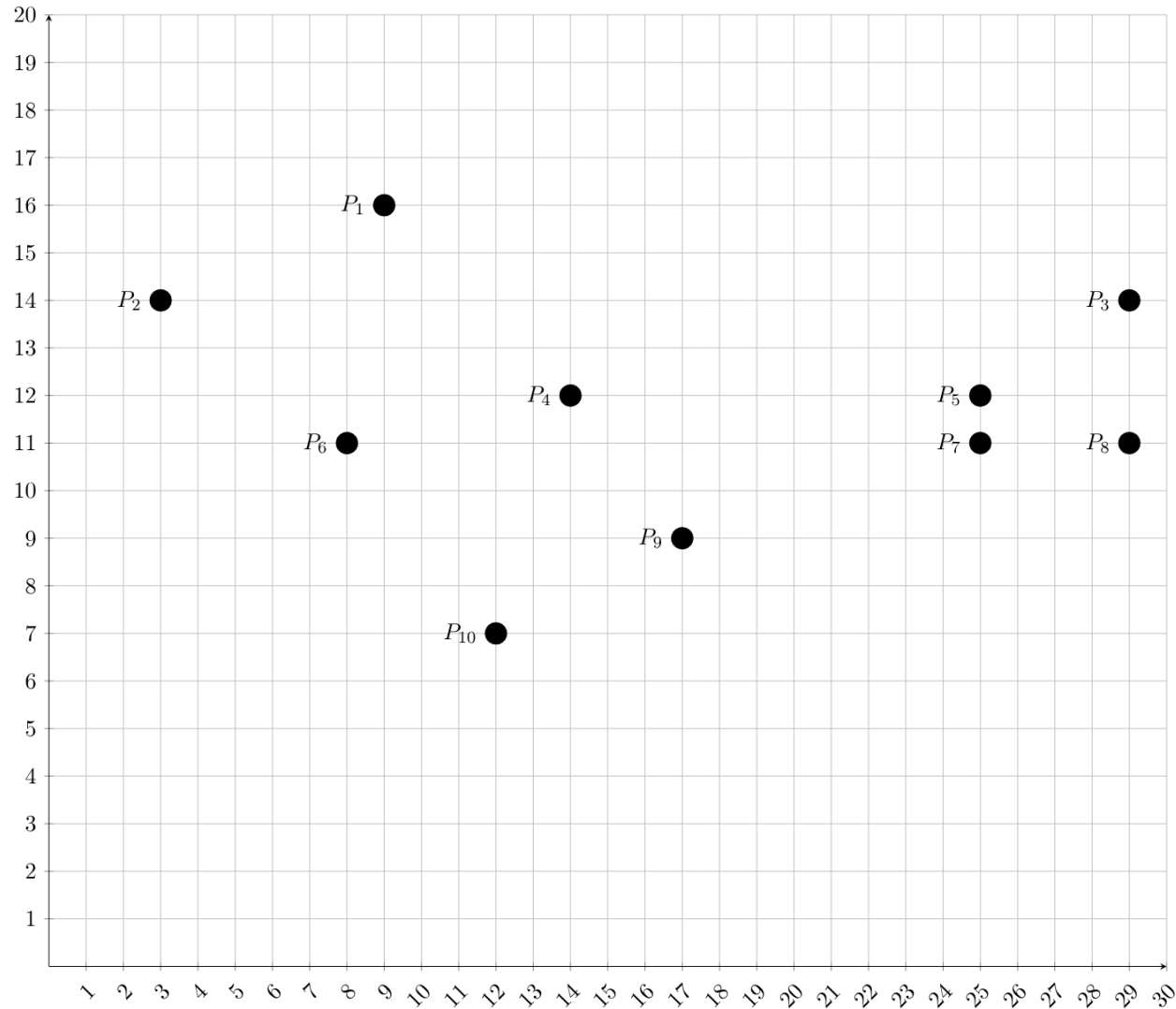
Zeichne die Aufteilung des Datenraums für QuadTrees auf.
Wähle immer die Mitte der Partition für die Aufteilung.

$$x \in [0, 30]; y \in [0, 20]$$

Einen Überlauf gibt es bei mehr als 2 Punkten in einer Partition.

Wenn ein Punkt auf einer Partitionslinie liegt, soll er der rechtesten und obersten Partition zugeordnet werden.

Folgendes Koordinatensystem ist gegeben:

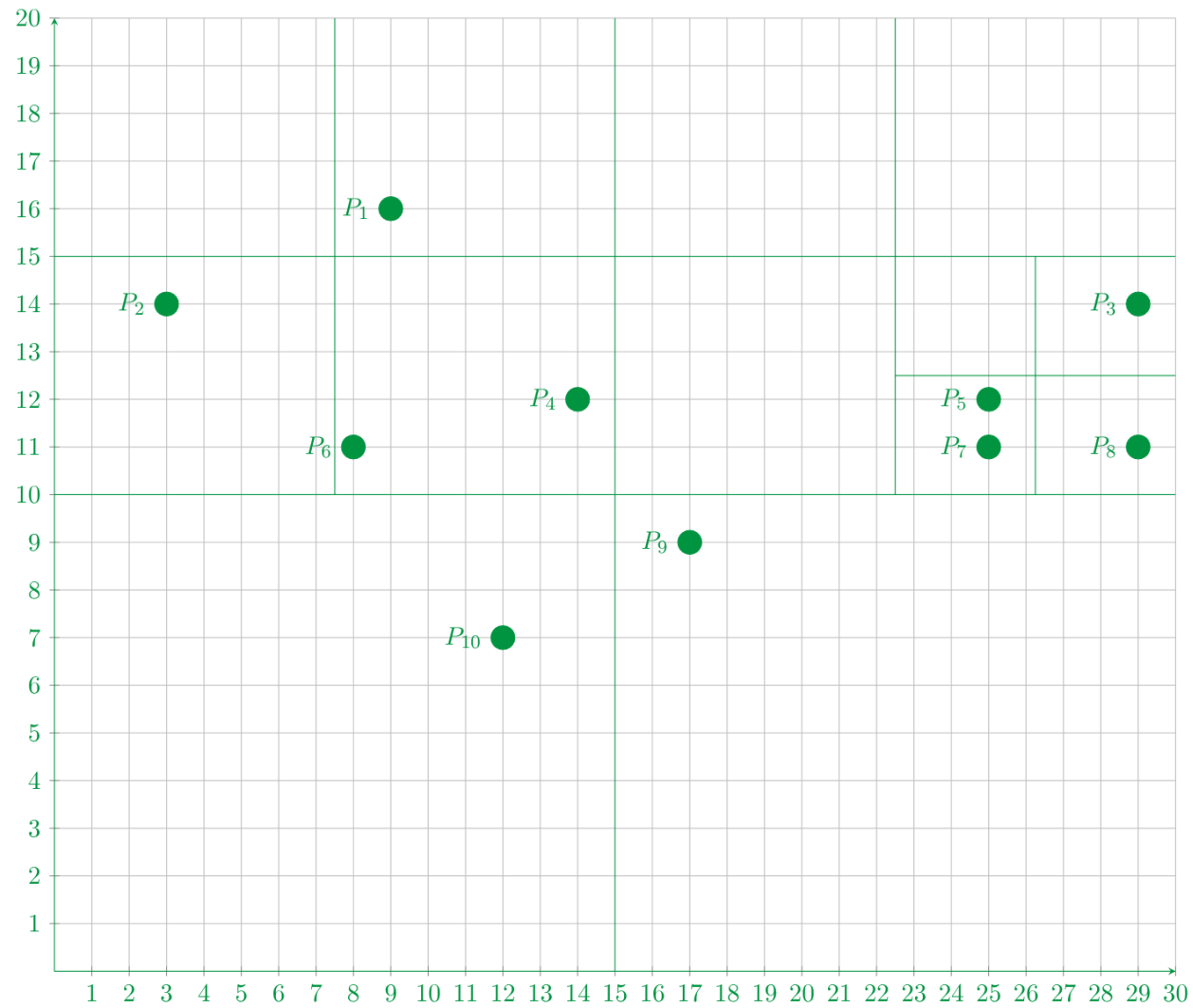


- Erste Trennung:
 - $x = \frac{0+30}{2} = 15$
 - $y = \frac{0+20}{2} = 10$
 - Überlauf: links oben und rechts oben

- Zweite Trennung:
 - Trennung links oben:
 - $x = \frac{0+15}{2} = 7,5$
 - $y = \frac{10+20}{2} = 15$
 - Trennung rechts oben:
 - $x = \frac{15+30}{2} = 22,5$
 - $y = \frac{10+20}{2} = 15$
 - Überlauf: rechts unten in der oberen rechten Partition

- Dritte Trennung:
 - Trennung rechts unten:
 - $x = \frac{22,5+30}{2} = 26,25$
 - $y = \frac{10+15}{2} = 12,5$

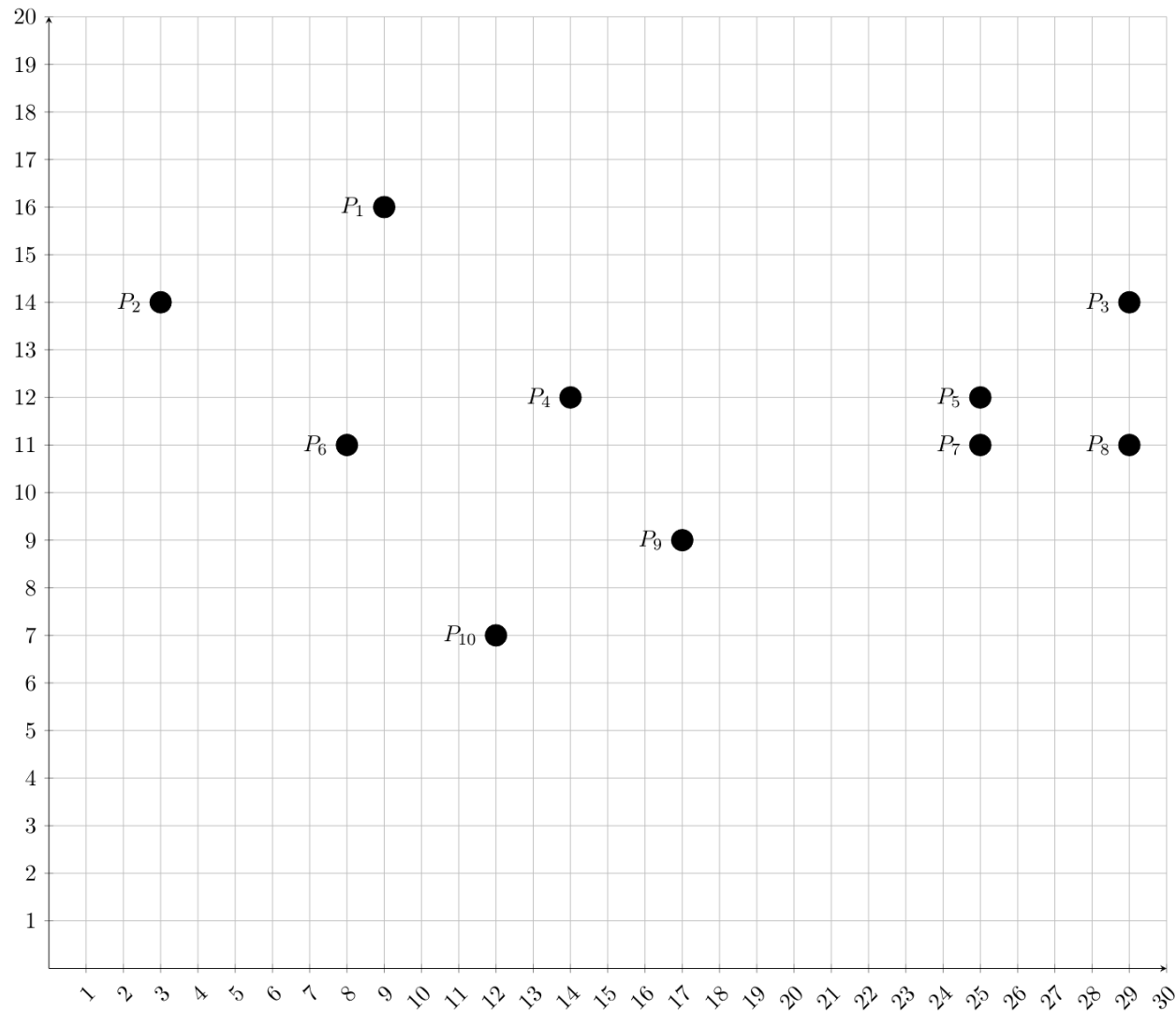
Ergebnis:



1 b)

Zeichne die Aufteilung des Datenraums für QuadTrees auf.
Wähle immer den Mittelwert aller Punkte in der Partition für die Aufteilung.

Folgendes Koordinatensystem ist gegeben:



- Erste Trennung:

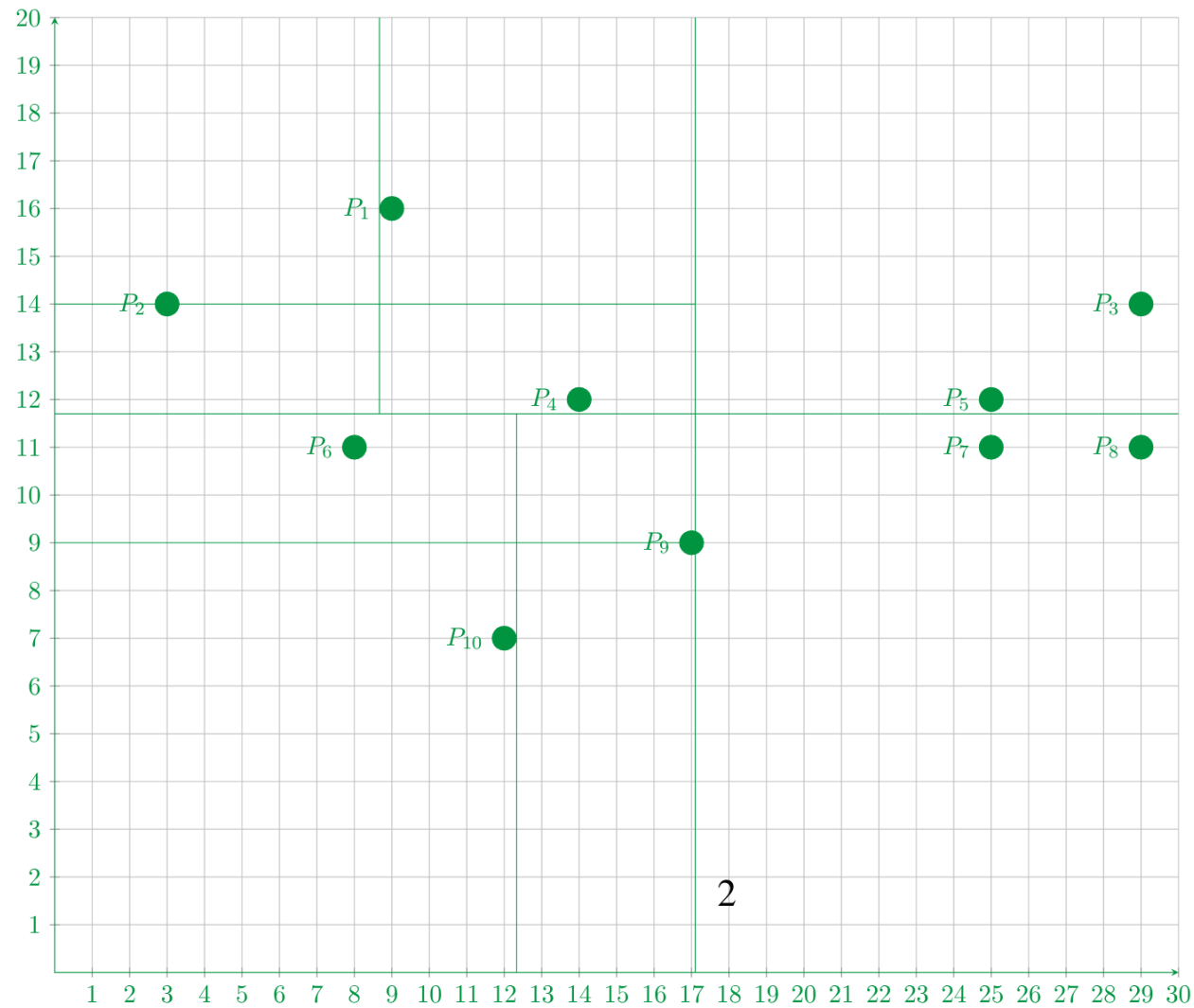
- $x = \frac{9+3+29+14+25+8+25+29+17+12}{10} = \frac{171}{10} = 17,1$

- $y = \frac{16+14+14+12+12+11+11+11+9+7}{10} = \frac{117}{10} = 11,7$

- Überlauf: oben links und unten links

- Zweite Trennung:
 - Trennung oben links:
 - $x = \frac{9+3+14}{3} = 8\frac{2}{3}$
 - $y = \frac{16+14+12}{3} = 14$
 - Trennung unten links:
 - $x = \frac{8+17+12}{3} = 12\frac{1}{3}$
 - $y = \frac{11+9+7}{3} = 9$

Ergebnis:



Wiederholung B-Baum

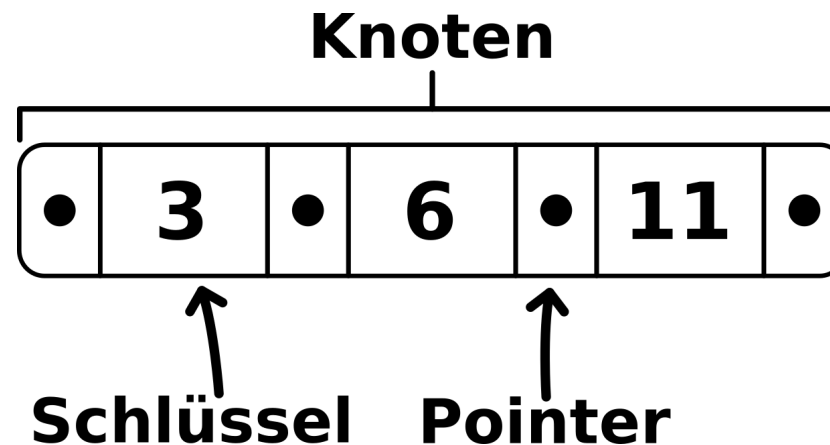
B-Baum

- ein B-Baum ist ein Mehrwege-Suchbaum, der Schlüssel mit Werten enthält
 - er speichert Daten in allen Knoten, sowohl in inneren Knoten als auch in Blättern
- er ist balanciert (alle Blätter haben die selbe Distanz zur Wurzel)

Begriffe:

- Ordnung:
 - jeder B-Baum hat eine Ordnung k , die bei $k \geq 1$ den Füllgrad vorgibt
 - Wurzel: minimal 1, maximal $2k$ Schlüssel
 - Knoten: minimal k , maximal $2k$ Schlüssel

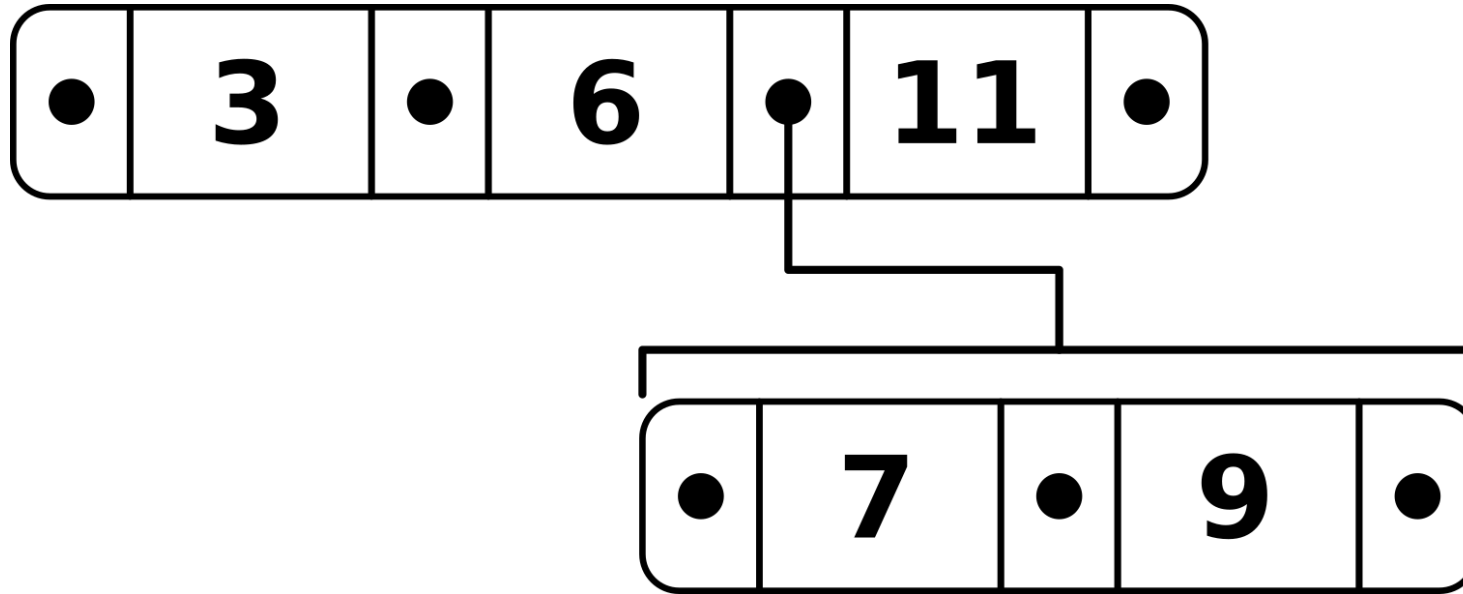
- Knoten:
 - ein Knoten enthält Schlüssel $s_1, \dots, s_l$, die sortiert sind
 - zwischen jedem Schlüsselpaar s_i und s_{i+1} gibt es einen Pointer
 - vor dem ersten und hinter dem letzten Schlüssel s_1 und s_l liegen auch Pointer



Beispiel: Knoten

- Pointer:
 - ein Pointer ist ein Zeiger, der bei inneren Knoten auf Nachfolgeknoten zeigt
 - bei Blättern zeigt der Pointer nirgendwo hin

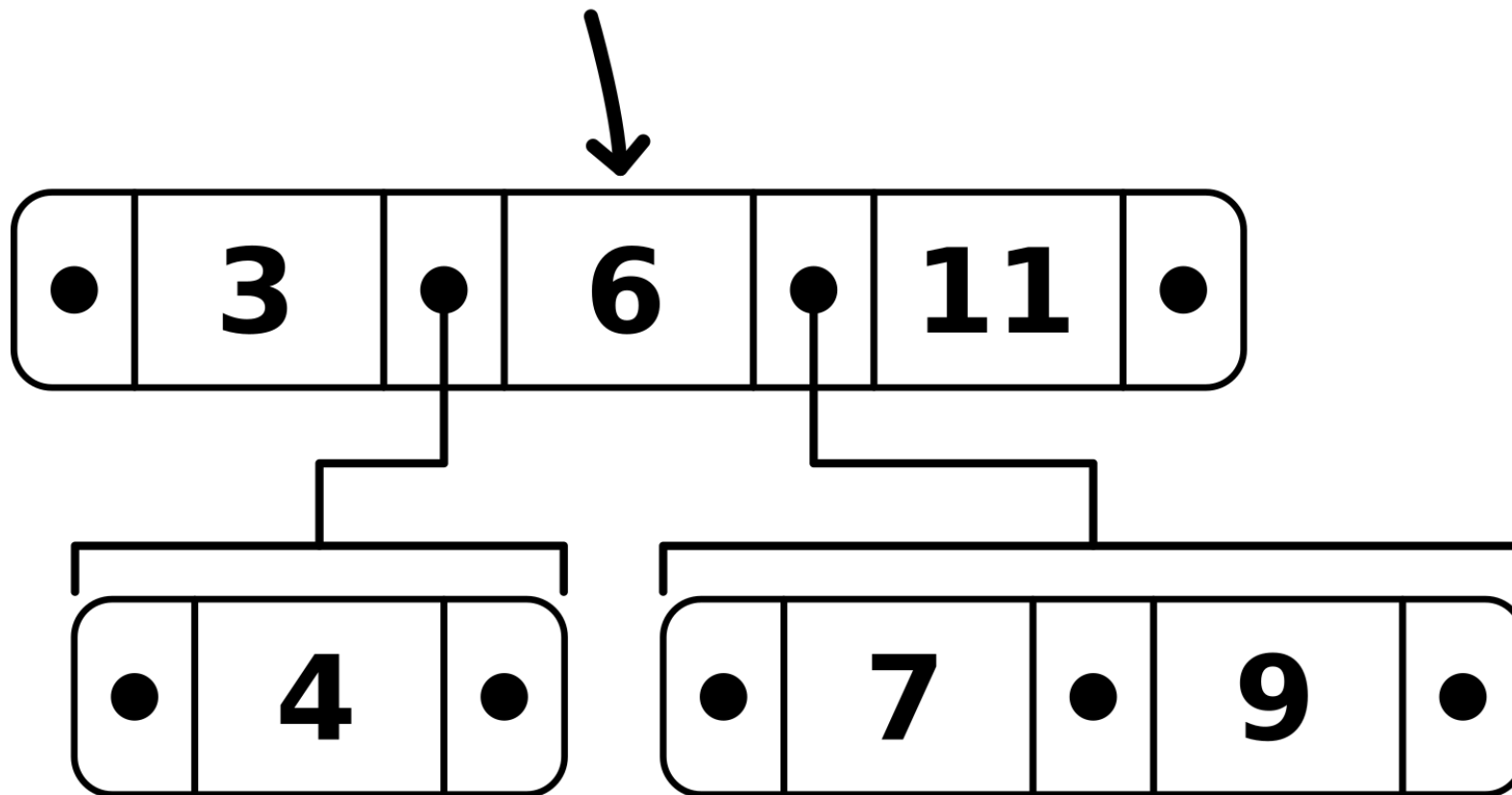
- Nachfolger:
 - ein Nachfolger ist der Knoten, auf den ein Pointer aus einem inneren Knoten zeigt
 - bei l Schlüsseln hat ein innerer Knoten $l + 1$ Nachfolgeknoten
 - die Werte der Schlüssel eines Nachfolgeknotens liegen zwischen den Werten des Schlüsselpaars im inneren Knoten



Beispiel: Nachfolger

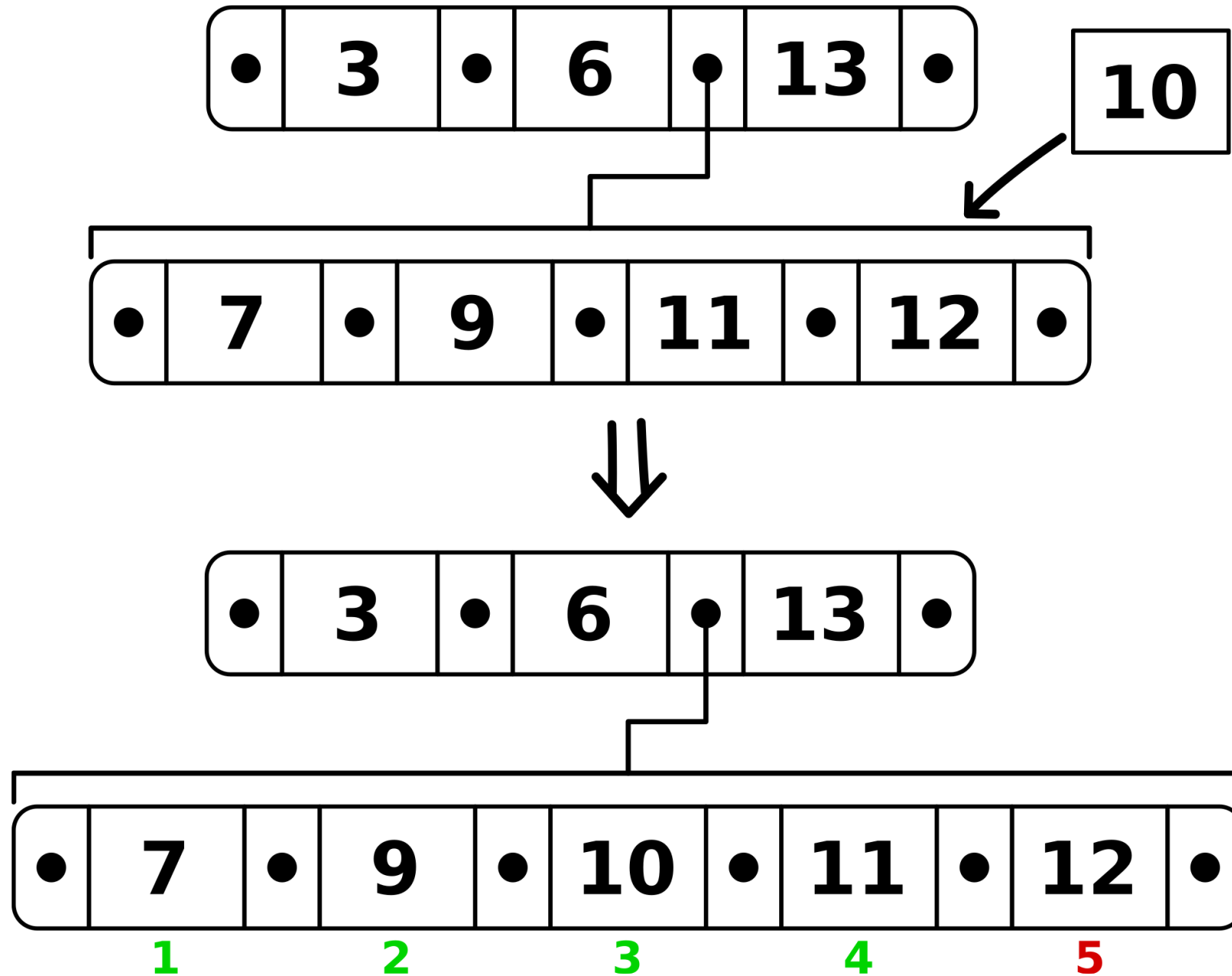
- Trennschlüssel:
 - der Schlüssel zwischen zwei Pointern wird Trennschlüssel genannt

Trennschlüssel



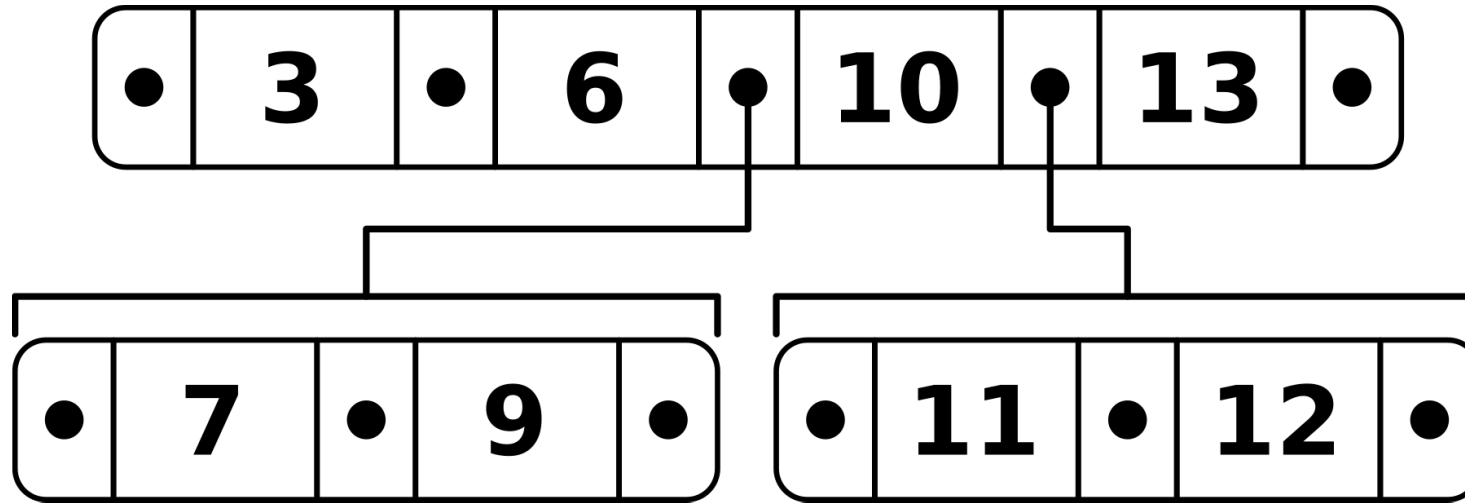
Element einfügen

- Suche das Blatt B , in das das neue Element eingefügt wird
- wenn nach dem Einfügen die Anzahl der Schlüssel kleiner als $2k$ ist, ist der Algorithmus fertig
- ist die Anzahl der Schlüssel größer als $2k$, muss ein Split durchgeführt werden:
 - Beispiel mit $k = 2$:



Beispiel: Einfügen 1

- Split:
 - ein neues Blatt B' wird erzeugt
 - der mittlere Schlüssel s_{k+1} des Blatts B kommt in den Vorgängerknoten als Trennschlüssel
 - die erste Hälfte der Schlüssel $s_1, \dots, s_k$ bleiben im Blatt B
 - die zweite Hälfte der Schlüssel $s_{k+2}, \dots, s_l$ kommen ins neue Blatt B'



Beispiel: Einfügen 2

Element löschen

- suche den Knoten B , in dem sich der Schlüssel s befindet, der gelöscht werden soll
- 2 Fälle:
 1. der Knoten B ist ein innerer Knoten
 - suche im linken Teilbaum von s den größten Schlüssel s'
 - ersetze s mit s'
 - lösche s' aus dem linken Teilbaum

2. der Knoten B ist ein Blatt

- lösche s aus B
- kommt es zum Unterlauf muss reorganisiert werden

Unterlauf nach dem Löschen

Ein Unterlauf kommt zustande, wenn die Anzahl der Schlüssel in einem Knoten B nach dem Löschen kleiner als die Ordnung k ist.

Bei einem Unterlauf gibt es drei Fälle:

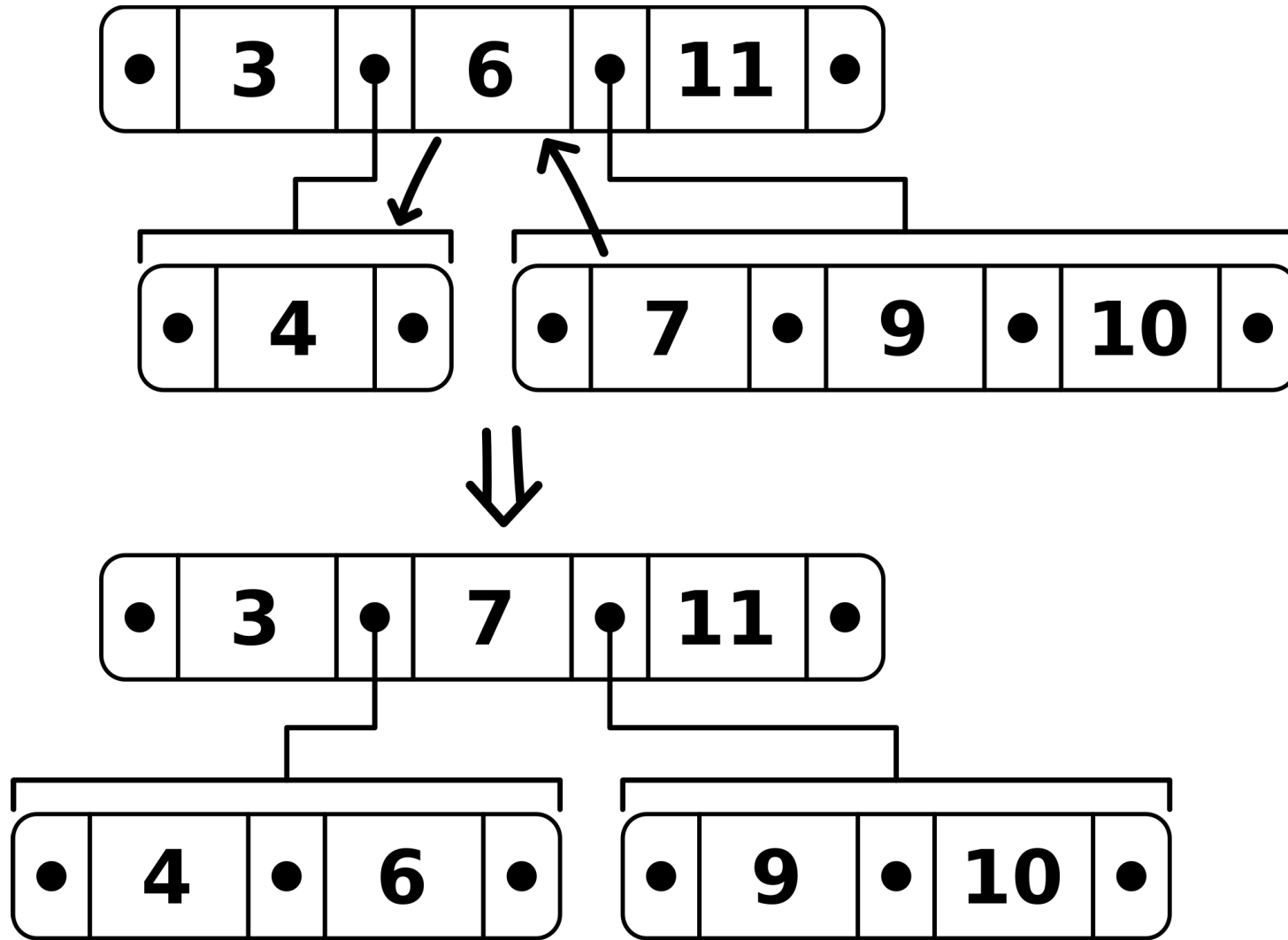
1. B ist die Wurzel

- die Wurzel darf weniger als k Schlüssel haben
- bei 0 Schlüsseln ist der Baum leer

2. Ausgleich: B hat einen Nachbarknoten B' mit mehr als k Schlüsseln

- der Trennschlüssel des Vorgängers kommt zu B
- wenn der Nachbarknoten B' rechts vom Knoten B liegt, wird der kleinste Schlüssel aus B' zum neuen Trennschlüssel im Vorgängerknoten
- wenn der Nachbarknoten B' links vom Knoten B liegt, wird der größte Schlüssel aus B' zum neuen Trennschlüssel im Vorgängerknoten

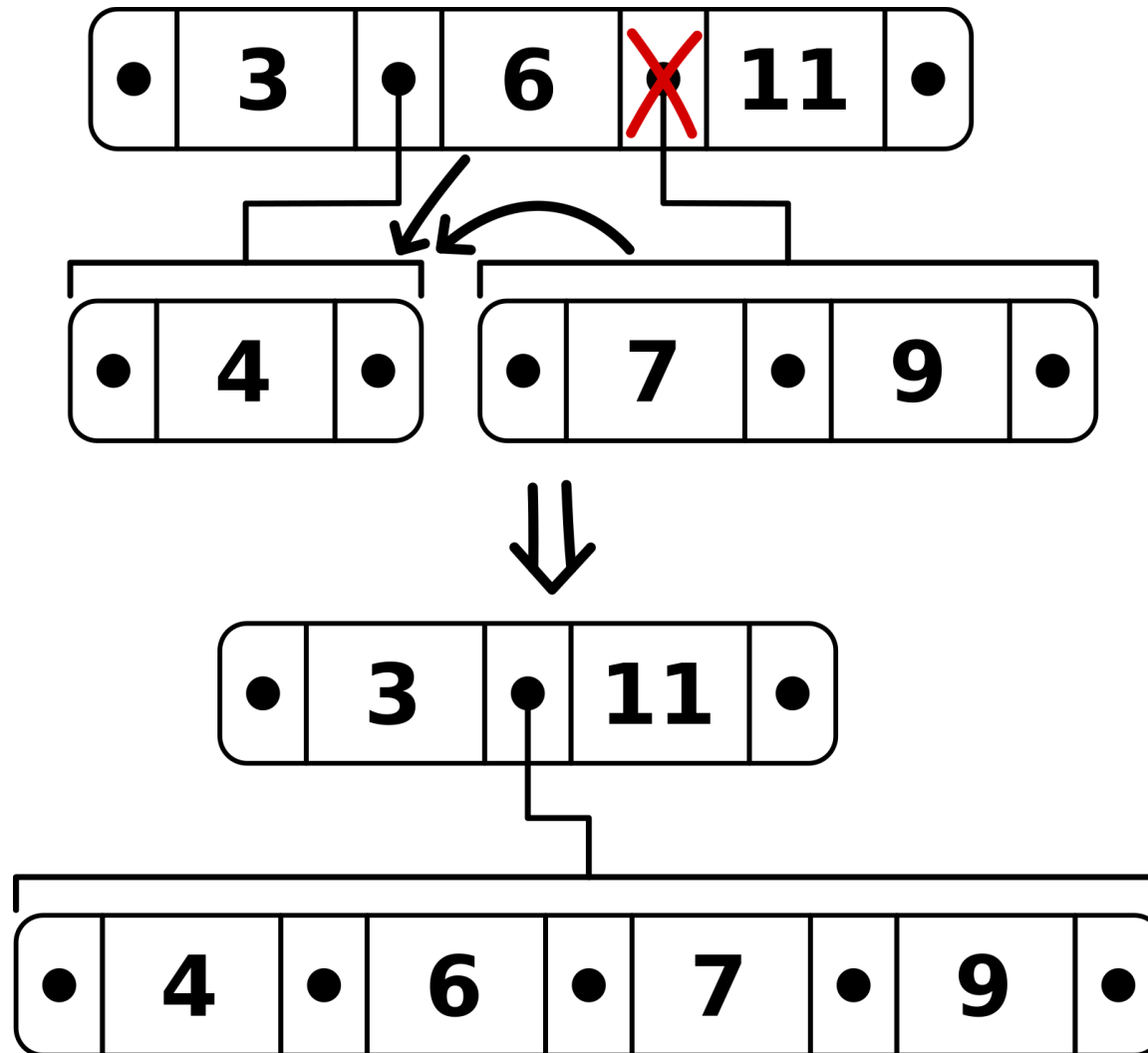
- Beispiel mit $k = 2$



Beispiel: Unterlauf 1

3. Verschmelzen: B hat einen Nachbarknoten B' mit genau k Schlüsseln

- der Trennschlüssel des Vorgängers sowie alle Schlüssel aus B' kommen nach B
- der Pointer zu B' im Vorgänger wird gelöscht und somit auch B'
- Beispiel mit $k = 2$



Beispiel: Unterlauf 2

Aufgabe 2

2 a)

Füge die Werte des Arrays A in einen leeren B-Baum mit $k = 1$ ein. Benutze bei einem Überlauf die Strategie des Splits.

$A = [16, 42, 89, 49, 35, 45, 8]$

- Einfügen: 16

16

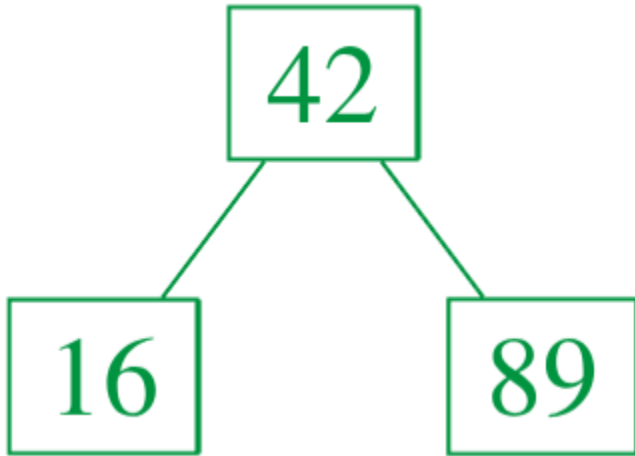
- Einfügen: 42

16	42
----	----

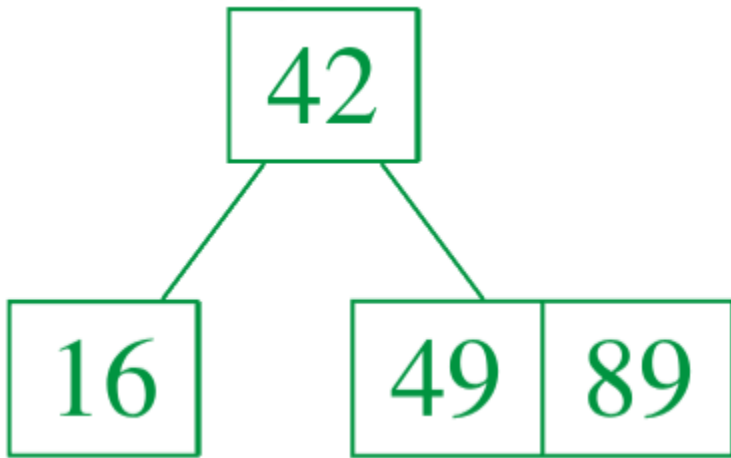
- Einfügen: 89

16	42	89
----	----	----

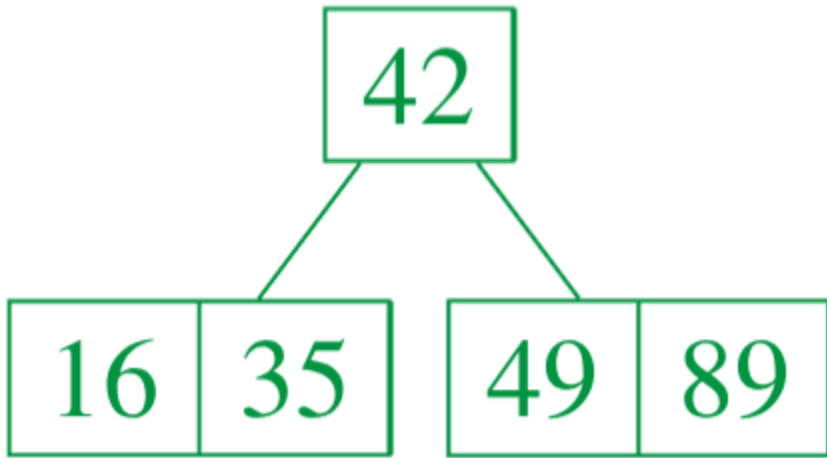
- Überlauf: Split



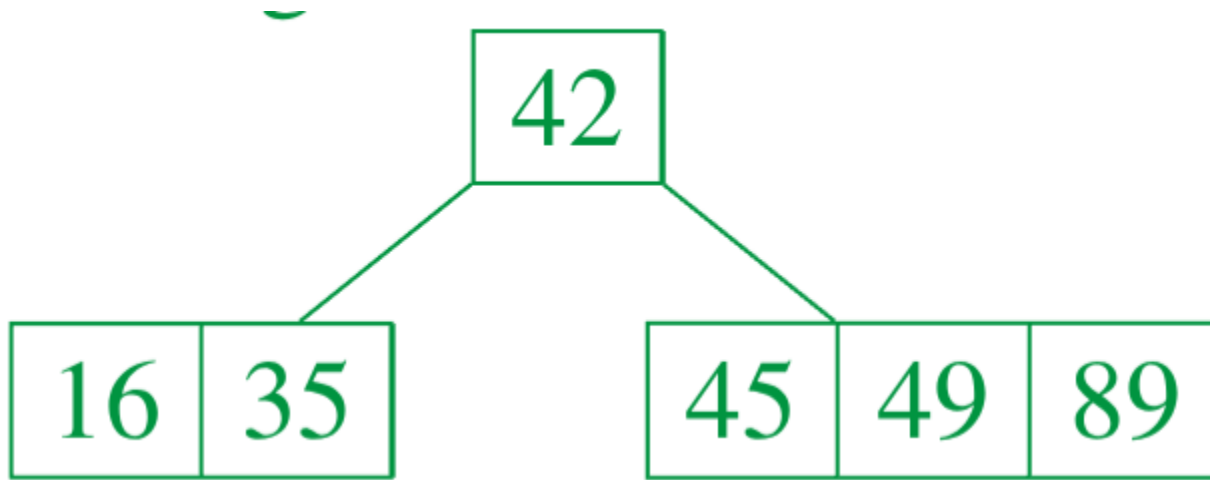
- Einfügen: 49



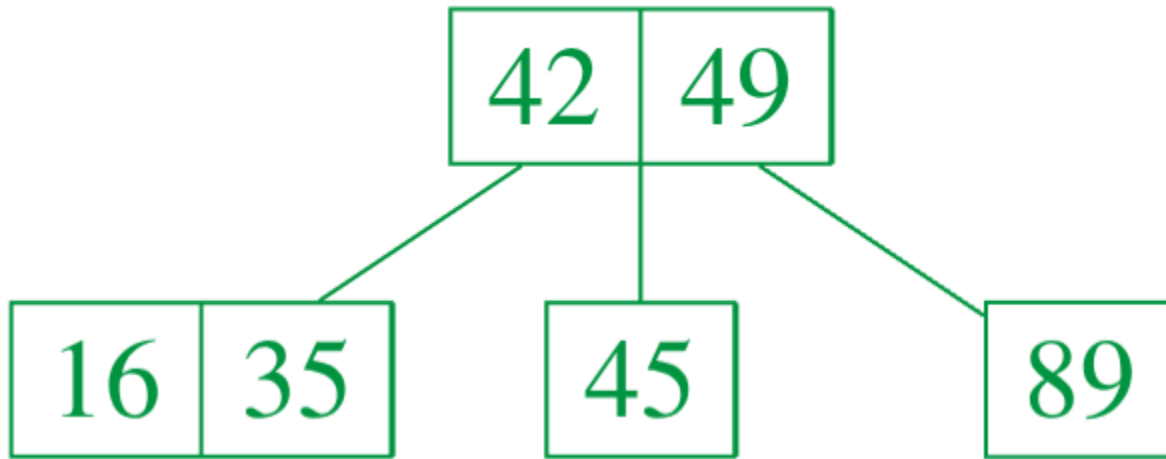
- Einfügen: 35



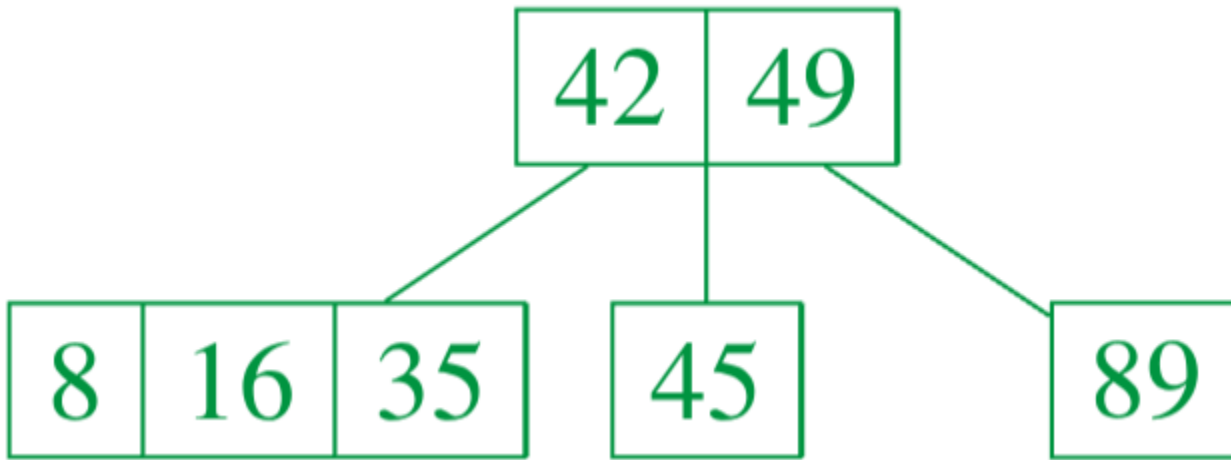
- Einfügen: 45



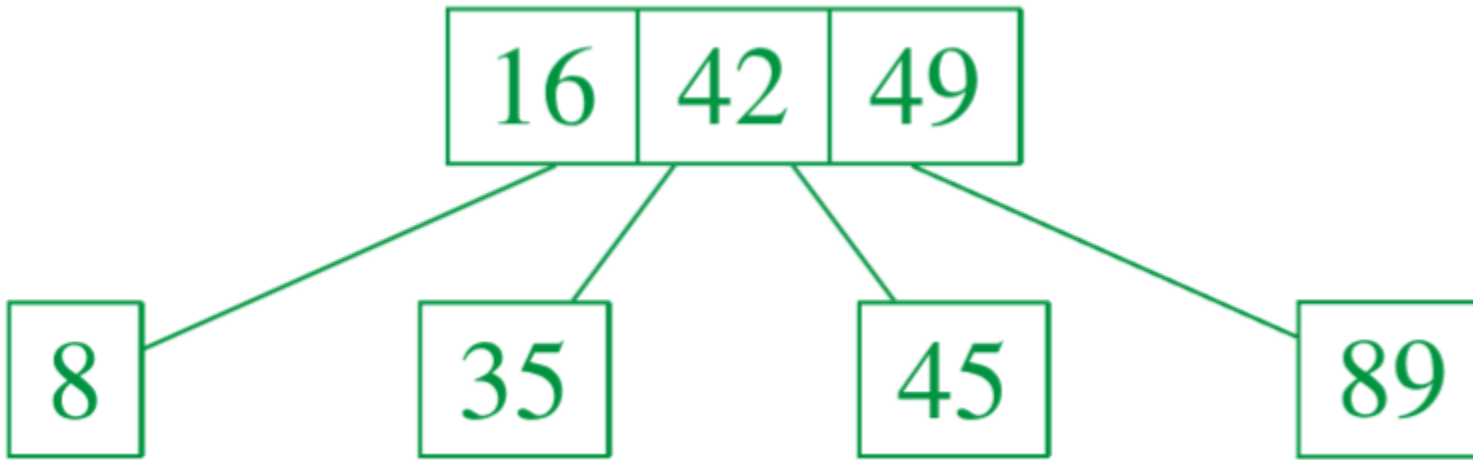
- Überlauf: Split



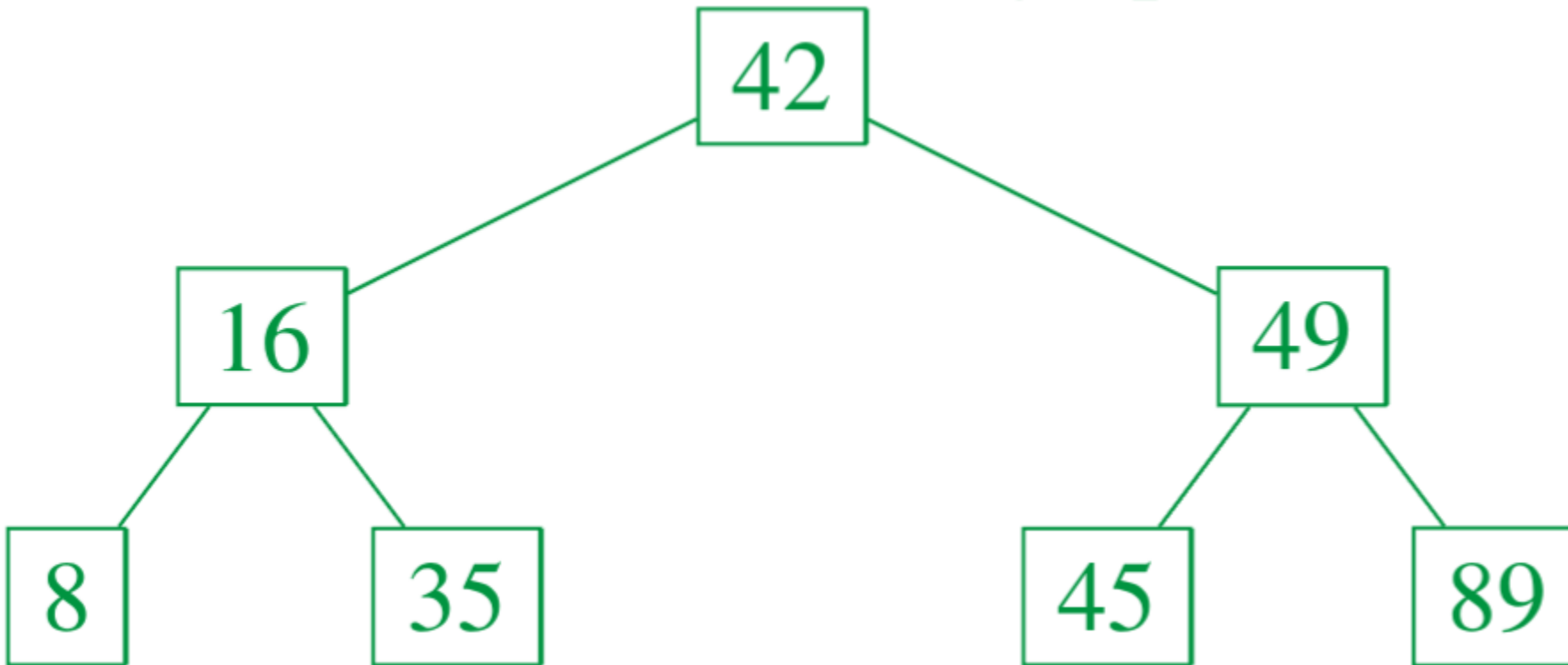
- Einfügen: 8



- Überlauf: Split

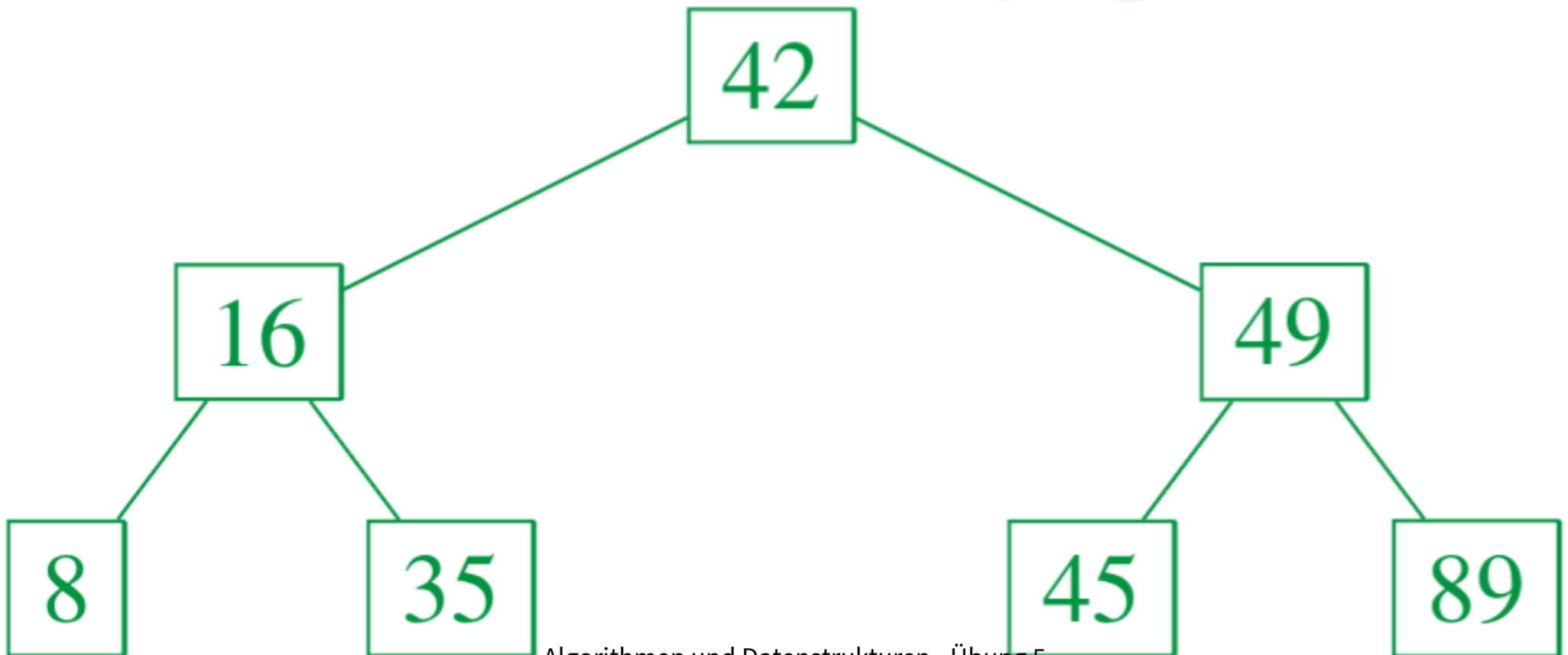


- Immernoch Überlauf: Split

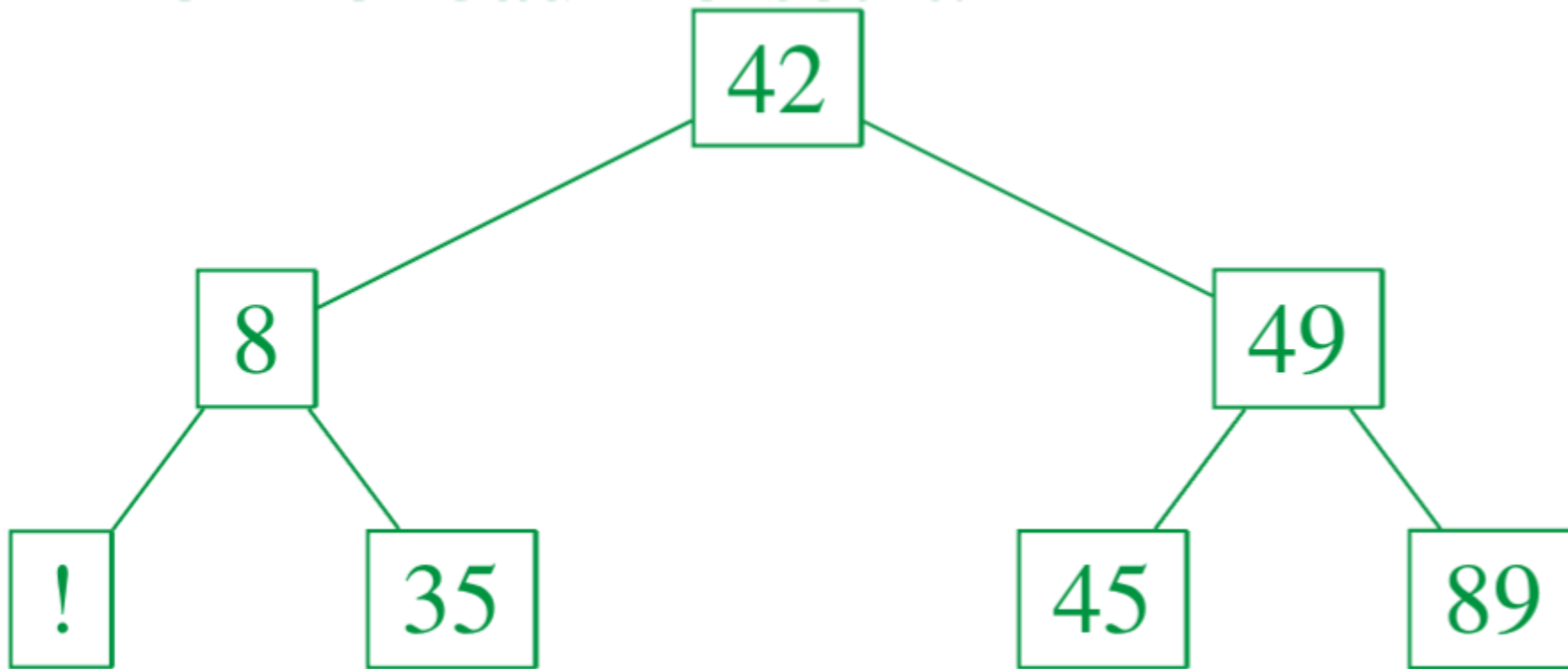


2b)

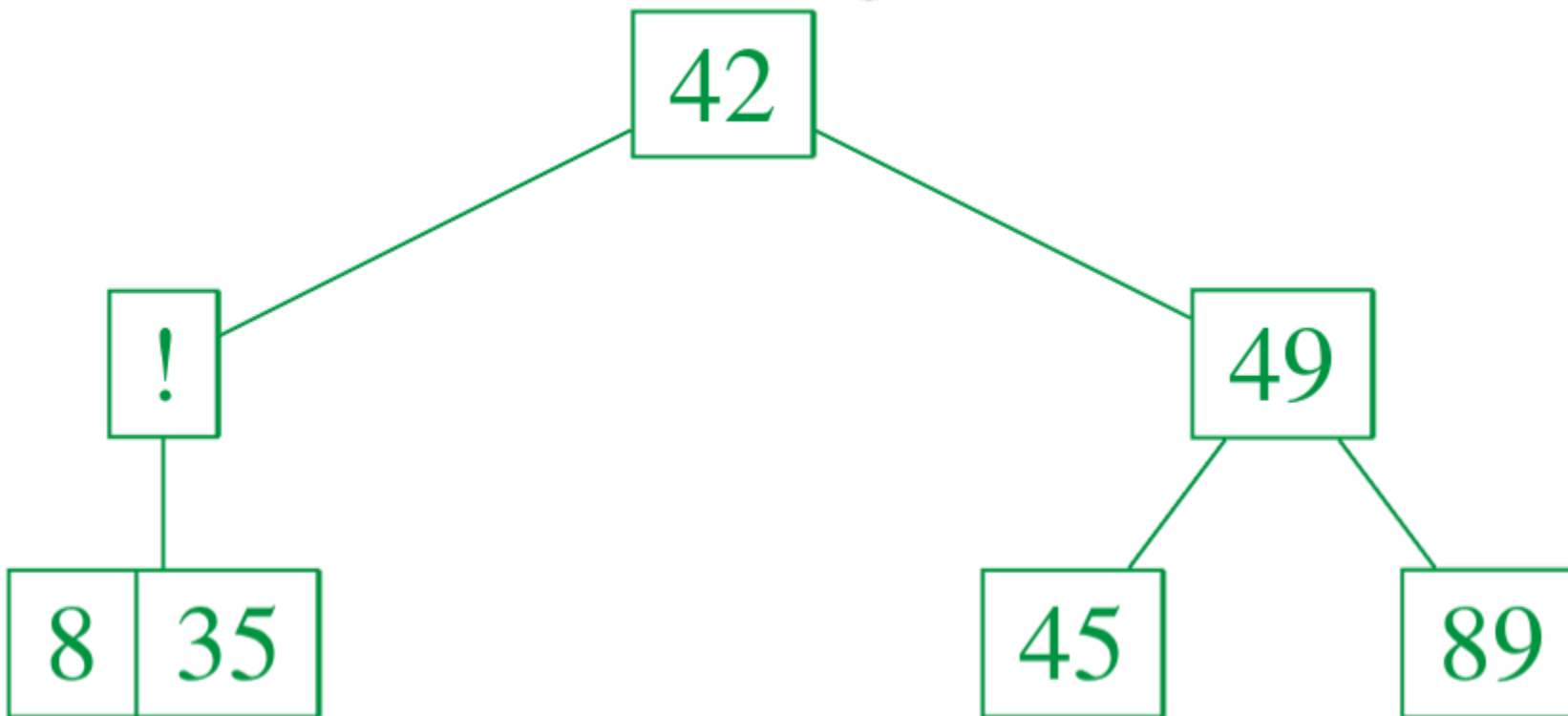
Lösche aus dem B-Baum mit der Ordnung $k = 1$ die Werte 16, 49, 89. Benutze bei einem Unterlauf die Strategie mit dem Nachbarknoten.



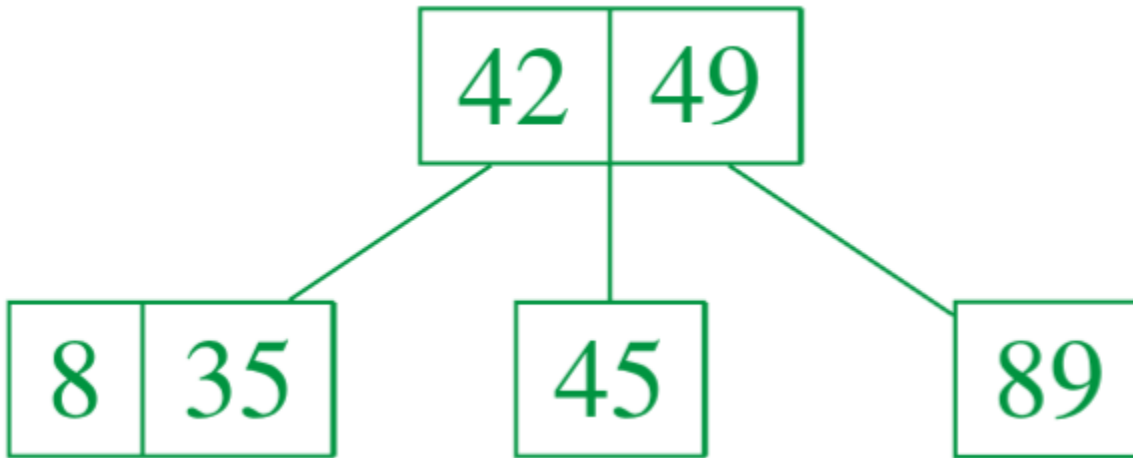
- Löschen: 16
 - 16 ist ein innerer Knoten $\rightarrow$ 1. Fall beim Löschen



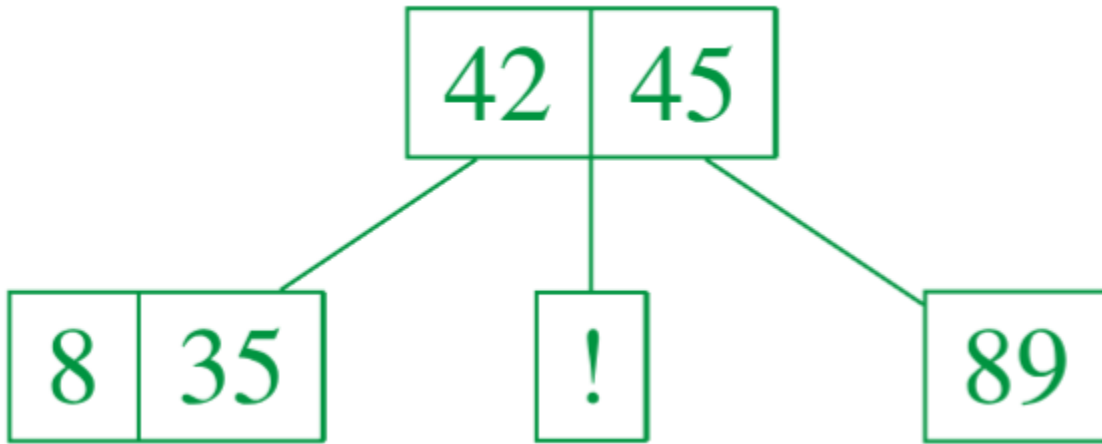
- Unterlauf:
 - Nachbarknoten hat k Schlüssel $\rightarrow$ Verschmelzen



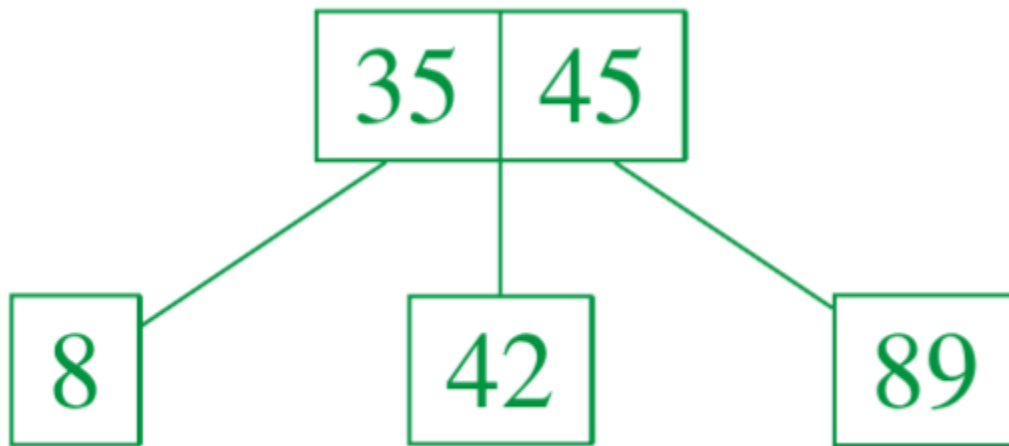
- nochmal Unterlauf:
 - Nachbarknoten hat k Schlüssel $\rightarrow$ wieder Verschmelzen



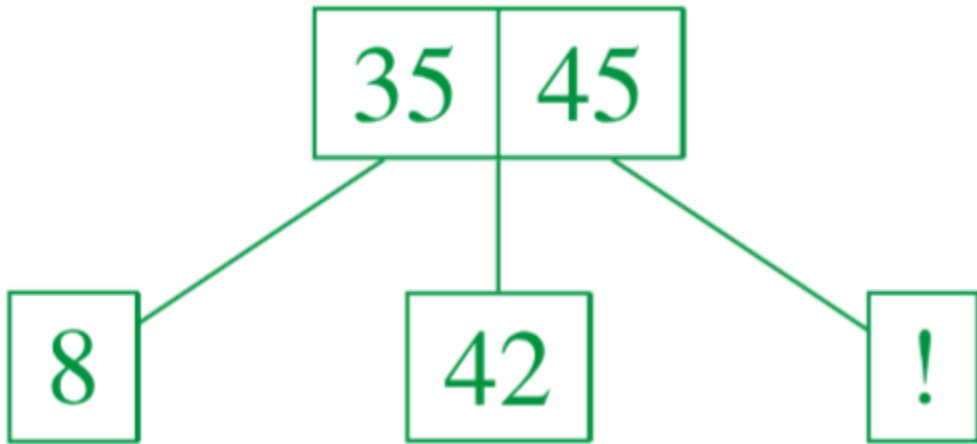
- Löschen: 49
 - 49 ist ein innerer Knoten (Wurzel) → 1. Fall beim Löschen



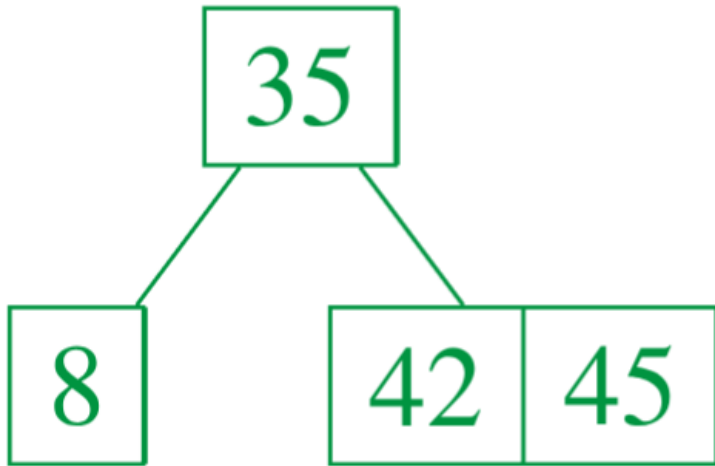
- Unterlauf:
 - Nachbarknoten (linker Knoten) hat mehr als k Schlüssel
→ Ausgleichen



- Löschen: 89
 - 89 ist ein Blatt $\rightarrow$ 2. Fall beim Löschen



- Unterlauf:
 - Nachbarknoten hat k Schlüssel $\rightarrow$ Verschmelzen



Wiederholung B+-Baum

B+-Baum

- ein B+-Baum ist eine Variante des B-Baums
 - er ist in der Praxis standardmäßig die Datenstruktur, die für fast alle Datenbank-Indizes verwendet wird
- im Gegensatz zum B-Baum speichert ein B+-Baum die Schlüssel und die zugehörigen Daten ausschließlich in den Blättern
 - die Blätter sind zusätzlich miteinander verkettet, um Bereichsanfragen effizient zu unterstützen

- die inneren Knoten enthalten nur Wegweiser, die dabei helfen, das richtige Blatt zu finden
 - als Trennschlüssel wird ein Schlüssel aus einem Blatt verwendet (Wegweiser = Kopie eines Schlüssels)
 - weil in den inneren Knoten nur Wegweiser und keine Daten gespeichert sind, passen viel mehr Wegweiser in einen inneren Knoten
 - das sorgt dafür, dass ein B+-Baum bei der gleichen Menge an Daten flacher sein kann als ein B-Baum

2c)

1. Wie kann ein analoger Algorithmus zum Einsortieren in einen B+-Baum aussehen?
2. Ordne das Array A in einen leeren B+-Baum mit $k = 1$ ein.

1.

Möglicher Algorithmus:

1. Die Elemente werden wie bei einem B-Baum in die Blätter eingefügt.
2. Aufteilung eines Blatts:
 - wenn ein Blatt mit den Schlüsseln $(x_1, \dots, x_{2k+1})$ aufgeteilt werden muss, werden zwei gleich große Knoten erstellt
 - in das eine kommen die Schlüssel $(x_1, \dots, x_k)$ und in das zweite kommen $(x_{k+1}, \dots, x_{2k+1})$

- im Elternknoten wird ein neuer Eintrag mit dem Wert x_{k+1} erstellt und die Pointer links und rechts vom neuen Eintrag zeigen auf die zwei neu erstellten Blätter
 - wichtig: der neue Schlüssel im Elternknoten ist eine Kopie des Schlüssels im Kindknoten
3. Aufteilung eines inneren Knotens: gleiche Vorgehensweise wie bei der Aufteilung beim B-Baum (Aufgabe 2a nach Einfügen des Elements 8)

2.

Elemente in B+-Baum mit $k = 1$ einfügen:

$A = [16, 42, 89, 49, 35, 45, 8]$

- Einfügen: 16

16

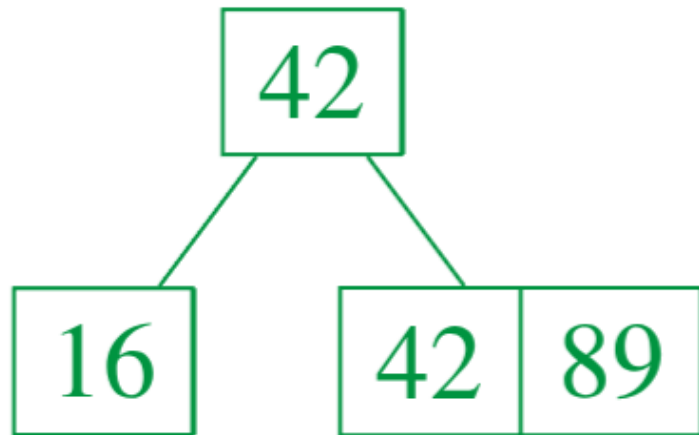
- Einfügen: 42

16	42
----	----

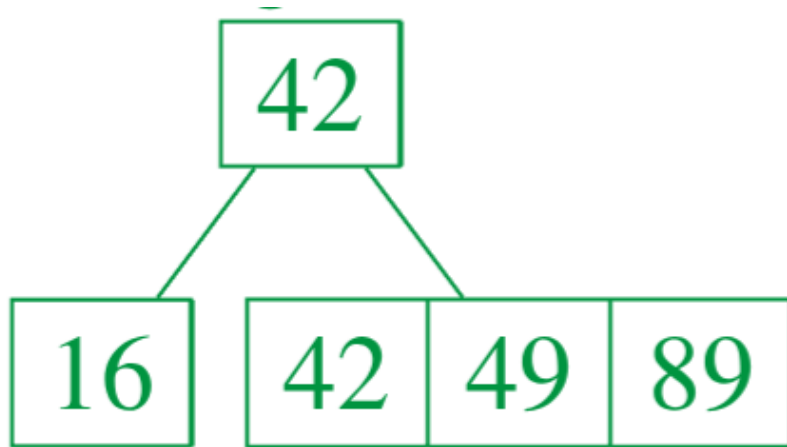
- Einfügen: 89



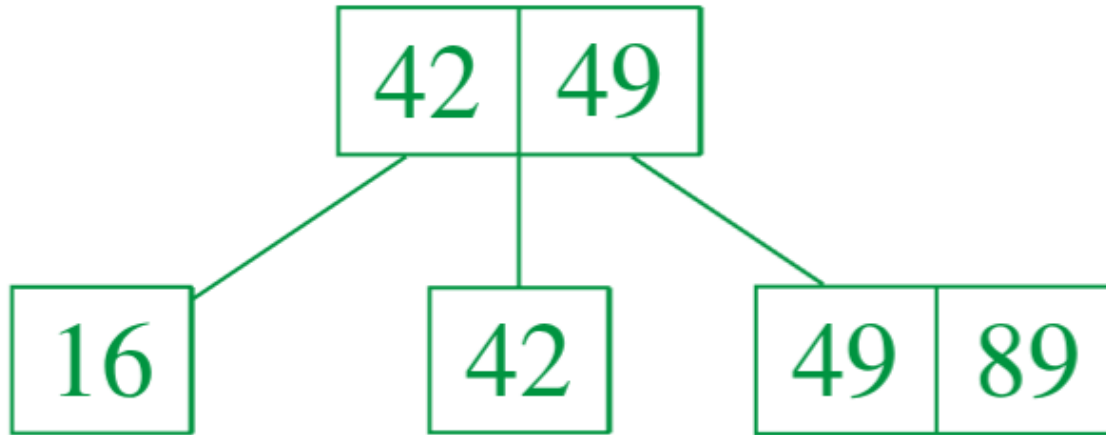
- Überlauf: Split



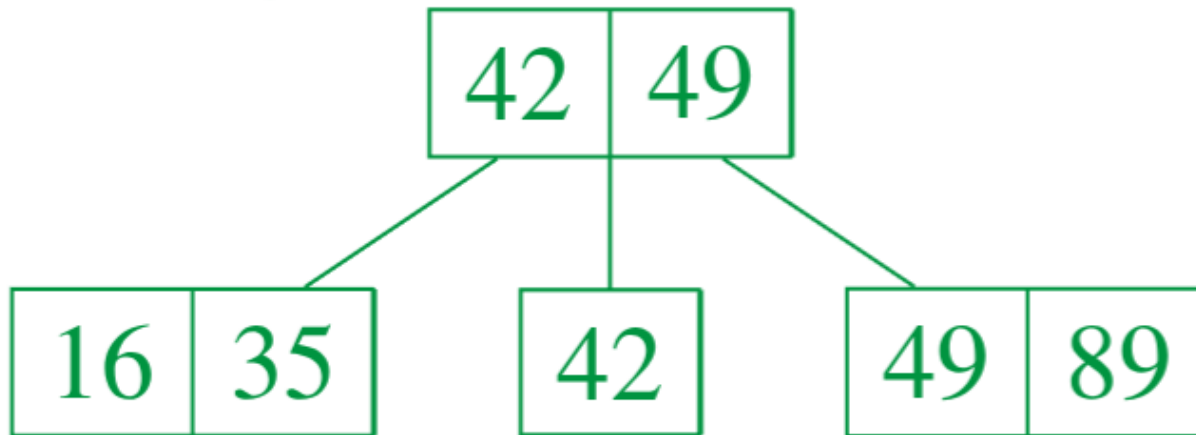
- Einfügen: 49



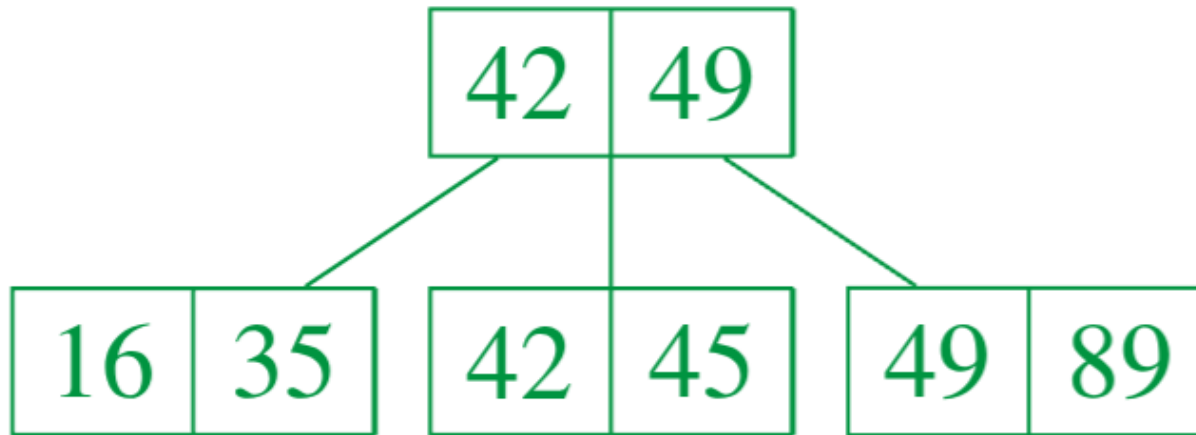
- Überlauf: Split



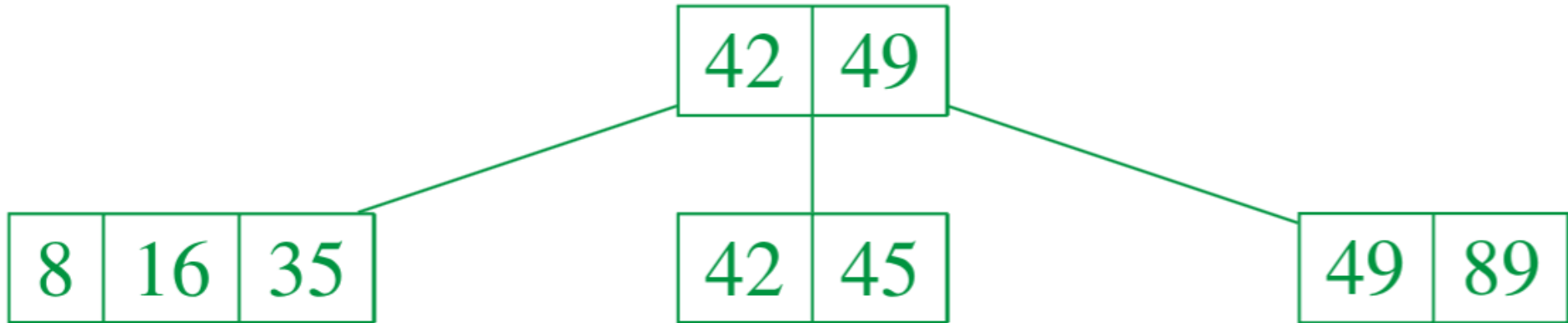
- Einfügen: 35



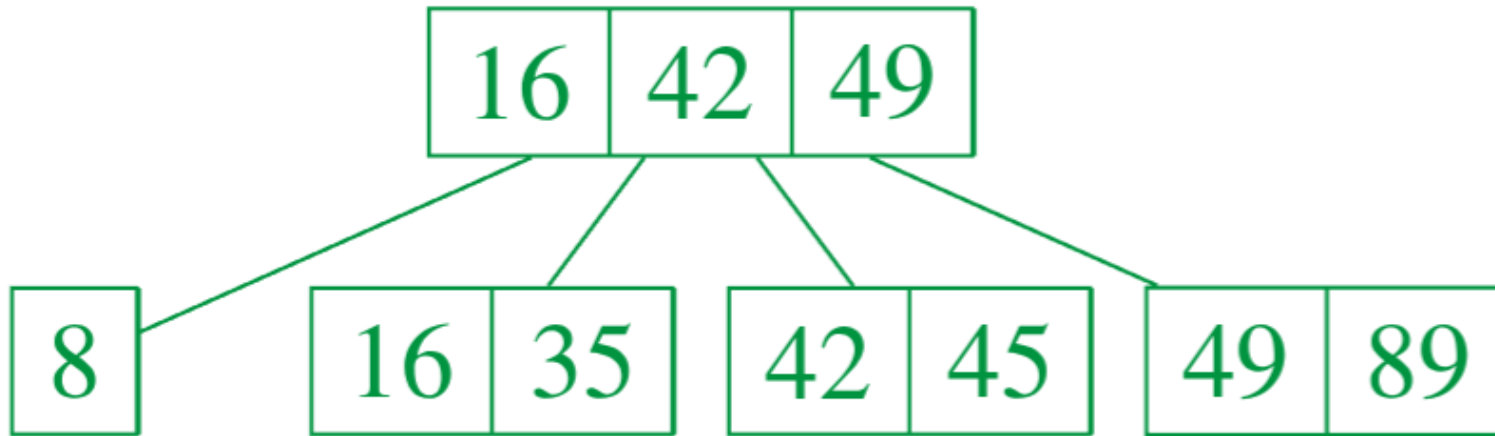
- Einfügen: 45



- Einfügen: 8



- Überlauf: Split



- Immernoch Überlauf: Split

